PHƯƠNG ÁN TRIỂN KHAI AI AGENT

1. MỤC TIÊU TRIỂN KHAI

Triển khai hệ thống Agent hỏi đáp nội bộ theo kiến trúc RAG, ưu tiên tiếng Việt, cho phép:

- Tiếp nhận dữ liệu từ nhiều nguồn như PDF, ảnh, docx, markdown, text, web crawler/API;
- Chuẩn hóa và phân loại dữ liệu trước khi lập chỉ mục;
- Truy xuất ngữ nghĩa theo mô hình parent-child chunk;
- Sinh câu trả lời có ngữ cảnh bằng local SLM;
- Tối ưu tốc độ phản hồi bằng semantic cache trên Redis;
- Bảo đảm khả năng mở rộng cho các nhóm tài liệu chuyên môn khác nhau của Tân Cảng Sài Gòn.

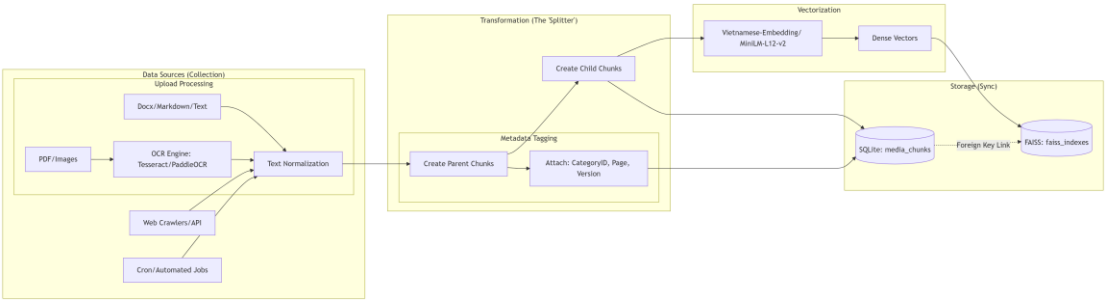
2. PHƯƠNG ÁN KĨ THUẬT

2.1. Kiến trúc tổng thể

Hệ thống được triển khai theo mô hình hybrid, kết hợp:

- Cloud API để thực hiện routing / HyDE / xử lý logic nhẹ;
- Local hosting để xử lý embedding, retrieval, generation và lưu trữ dữ liệu nội bộ.

Luồng kỹ thuật tổng thể gồm 4 lớp:



A. Thu thập và tiền xử lý dữ liệu

Hệ thống tiếp nhận dữ liệu từ nhiều nguồn như PDF, hình ảnh, docx, markdown, text, web crawler/API và các tác vụ tự động. Với file scan hoặc hình ảnh, hệ thống dùng OCR để trích xuất nội dung, sau đó chuẩn hóa văn bản để đưa về một định dạng thống nhất, sẵn sàng cho bước phân mảnh và lập chỉ mục.

B. Phân mảnh dữ liệu và gắn metadata

Sau khi chuẩn hóa, dữ liệu được chia thành 2 cấp: **parent chunk** là đoạn nội dung đầy đủ và **child chunk** là các đoạn nhỏ hơn để tối ưu tìm kiếm. Mỗi chunk được gắn metadata như nhóm tài liệu, số trang và phiên bản, giúp hệ thống tìm nhanh nhưng vẫn trả về đúng ngữ cảnh đầy đủ.

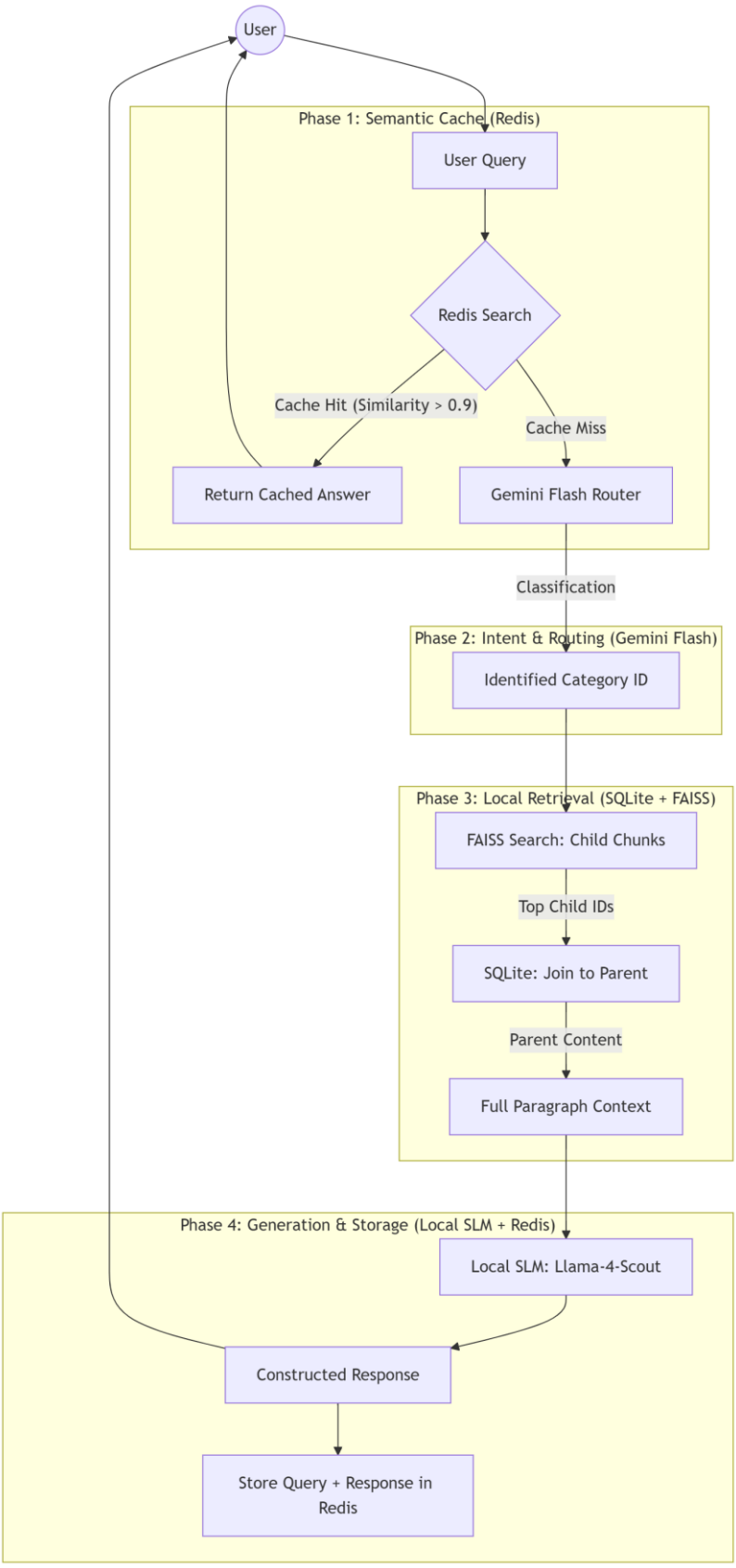
C. Vector hóa và lưu trữ

Các child chunk được chuyển thành vector bằng mô hình embedding tiếng Việt. Vector được lưu trong **FAISS** để phục vụ tìm kiếm ngữ nghĩa, còn nội dung và metadata được lưu trong **SQLite**. Hai phần này được liên kết với nhau để bảo đảm truy xuất chính xác.

D. Truy vấn và sinh câu trả lời

Khi người dùng đặt câu hỏi, hệ thống kiểm tra **Redis** trước để tận dụng các câu trả lời đã có nếu nội dung tương tự. Nếu không có, hệ thống dùng **Gemini Flash** để xác định đúng nhóm câu hỏi, sau đó tìm các đoạn liên quan trong **FAISS + SQLite**. Cuối cùng, ngữ cảnh

được đưa vào mô hình cục bộ để sinh câu trả lời và lưu lại vào Redis để tăng tốc cho các lần hỏi sau.



2.2. Chiến lược xử lý tiếng Việt và truy vấn dài

Do embedding model có giới hạn khoảng 512 tokens, tài liệu chiến lược đề xuất 3 hướng xử lý cần kiểm thử trong giai đoạn triển khai:

Phương án 1 – HyDE
Dùng Gemini Flash tạo "câu trả lời giả định" hoặc bản tóm tắt ngắn từ câu hỏi dài, sau đó mới embedding và search.
Phù hợp khi người dùng nhập mô tả dài, nhiều nhiễu, hoặc diễn đạt không theo từ khóa chuẩn.
Phương án 2 – Query Decomposition
Tách câu hỏi phức tạp thành nhiều câu hỏi đơn lẻ, rồi truy xuất song song theo từng ý.
Phù hợp với câu hỏi đa lớp như quy trình, trách nhiệm, thủ tục, biểu mẫu, chính sách nội bộ.
Phương án 3 – Sliding Window Embedding
Cắt truy vấn dài thành các đoạn có overlap, embedding từng đoạn rồi hợp nhất kết quả.
Phù hợp khi muốn giảm phụ thuộc vào LLM bên ngoài, hoặc cần fallback cục bộ.

Đề xuất áp dụng cho TCSG:
- Mặc định dùng **HyDE** cho query dài;
- Dùng **Query Decomposition** cho các câu hỏi có nhiều vế nghiệp vụ;
- Giữ **Sliding Window** như fallback kỹ thuật.

Cách này bám sát tài liệu chiến lược và phù hợp với đặc thù hỏi đáp tài liệu tiếng Việt, nơi câu hỏi thường dài, mô tả theo ngữ cảnh thực tế thay vì từ khóa ngắn.

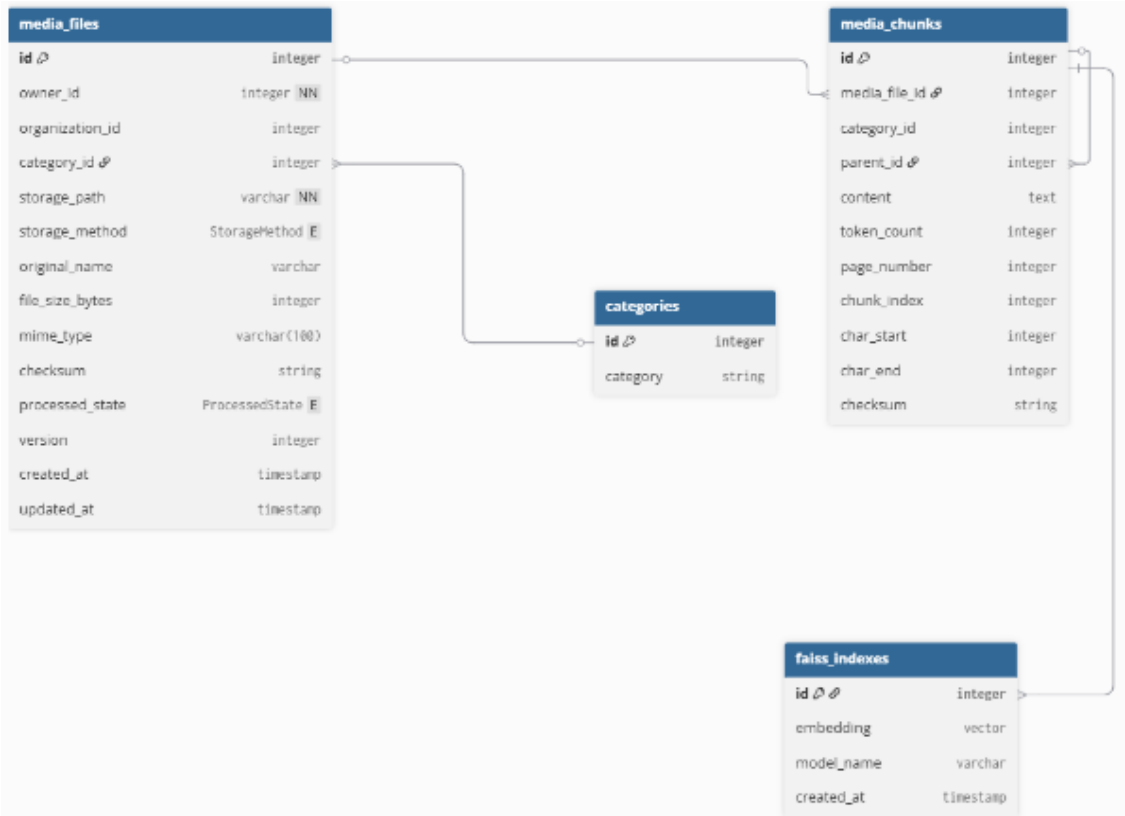
2.3. Tech stack đề xuất
Mô hình / AI
- **Router / HyDE / phân loại:** Gemini Flash
- **Embedding:** vietnamese-embedding
- **Local generation:** Llama-4-Scout bản quantized 4-bit.

Dữ liệu / lưu trữ
- **Vector store:** FAISS
- **Metadata & content store:** SQLite
- **Cache:** Redis.

Tiền xử lý
- **OCR:** Tesseract hoặc PaddleOCR
- **Normalization:** custom pipeline Python
- **Automated ingestion:** cron jobs / crawler / API sync.

Logic triển khai
- Python service cho ingestion, chunking, embedding, retrieval
- config quản trị tập trung qua config.yaml
- có thể đóng gói thành service riêng cho từng lớp: ingestion, indexing, query service, cache service.

2.4. Cấu trúc dữ liệu đề xuất
Theo ERD đính kèm, hệ thống nên duy trì tối thiểu 4 bảng chính:



3. TIMELINE TRIỂN KHAI ĐỀ XUẤT

Từ timeline nền đã có trong tài liệu, tôi đề xuất chuẩn hóa thành kế hoạch 5 giai đoạn để phù hợp hơn với triển khai thực tế cho Tân Cảng Sài Gòn. Mốc thời gian vẫn bám theo khung hiện có trong tài liệu gốc, nhưng được bổ sung rõ đầu việc, đầu ra và điểm nghiệm thu.

Giai đoạn 1 – Xây dựng hạ tầng dữ liệu và pipeline xử lý: 5 ngày làm việc

- Thiết lập môi trường kỹ thuật
- Xây dựng pipeline ingest, OCR, chuẩn hóa dữ liệu, chunking, metadata tagging
- Khởi tạo SQLite, FAISS, Redis và kiểm thử dữ liệu mẫu

Giai đoạn 2 – Tích hợp routing, semantic cache và retrieval: 8 ngày làm việc

- Tích hợp Gemini Flash cho phân loại và routing
- Thiết lập semantic cache trên Redis
- Triển khai cơ chế retrieval giữa FAISS và SQLite
- Kiểm thử các phương án xử lý query dài

Giai đoạn 3 – Tích hợp mô hình sinh câu trả lời và tối ưu hệ thống: 8 ngày làm việc

- Tích hợp local SLM
- Tối ưu prompt, tốc độ phản hồi và chất lượng câu trả lời
- Kiểm thử theo bộ câu hỏi thực tế

Giai đoạn 4 – UAT, hiệu chỉnh và golive nội bộ: 4 ngày làm việc

- UAT với nhóm người dùng đại diện
- Hiệu chỉnh hệ thống theo phản hồi
- Hoàn thiện tài liệu vận hành và chuẩn bị golive

Tổng thời gian triển khai: 27 ngày làm việc